

第六章：文件系统与I/O重定向

引言

本章导读

文件的最早起源于我们需要将数据长久保存在存储设备中的需求。

大家不用被“持久存储设备”这个术语吓到，它其实指的是计算机发展过程中使用的各类存储介质。在早期，包括卡片、纸带、磁芯、磁鼓等；而至今仍在使用有磁带、磁盘、硬盘，以及近年逐渐普及的U盘、闪存、固态硬盘（SSD）等。这些设备通常被称为“外存”。在此之前，我们主要接触的是另一种存储——内存（也叫RAM）。与内存相比，持久存储设备的读写速度较慢，但容量通常更大。最关键的区别在于：内存断电后数据会丢失，而外存在断电后仍能保留数据。因此，将需要长期保存的数据从内存写入外存，或者从外存加载到内存，是应用程序和操作系统必须具备的功能。

注释：文件系统在UNIX操作系统中具有特殊地位。根据《UNIX: A History and a Memoir》记载，1969年，Ken Thompson（UNIX的作者）在贝尔实验室工作期间，为了测试自己为PDP-7计算机编写的磁盘调度算法，开始编写一个批量读写数据的测试程序。在开发过程中，他逐渐意识到这个程序可以扩展为一个文件系统，进而发展成一个完整的操作系统。他当时直觉认为，实现一个操作系统只需要三周时间：第一周编写代码编辑器，第二周编写汇编器，第三周编写shell程序，并在同时逐步添加操作系统所需的功能（如exec等系统调用）。三周后，为测试磁盘调度算法而诞生的UNIX雏形便形成了。

本章我们将实现一个简单的文件系统——easyfs，用于管理持久存储设备这类I/O资源。为了支持应用程序访问持久存储设备，内核需要引入两种文件类型：常规文件和目录文件。它们都以文件系统所维护的磁盘文件的形式，被组织并保存在持久存储设备上。

同时，随着对文件这一抽象概念的实现逐步完善，我们也更容易体现UNIX“一切皆文件”的重要设计理念。我们将扩展与程序执行相关的exec系统调用，增加对运行参数的支持，并进一步改进shell程序的实现，使其能够识别和处理重定向符号“>”和“<”。这样一来，我们就能像UNIX中的shell一样，基于文件机制实现灵活的I/O重定向和管道操作，从而更灵活地将多个应用程序组合起来，完成复杂的功能。

实践体验

获取本章代码：

```
$ git checkout ch6
```

在 qemu 模拟器上运行本章代码：

```
$ make test BASE=1
>> ch6b_usertest
>> ch6b_filetest_simple
file_test passed!
Shell: Process 2 exited with code 0
>>
```

它会将 `Hello, world!` 输出到另一个文件 `filea`，并读取里面的内容确认输出正确。我们也可以通过命令行工具 `ch6b_cat` 来查看 `filea` 中的内容：

```
>> ch6b_cat
Hello, world!
Shell: Process 2 exited with code 0
>>
```

本章代码树

```
.
├── bootloader
│   └── rustsbi-qemu.bin
├── LICENSE
├── Makefile
├── nfs (新增, 辅助程序, 要来将 .bin 打包为 os 可以识别的文件镜像)
│   ├── fs.c
│   ├── fs.h
│   ├── Makefile
│   └── types.h
├── os
│   ├── bio.c (新增, IO buffer 的实现)
│   ├── bio.h
│   ├── console.c
│   ├── console.h
│   ├── const.h
│   ├── defs.h
│   ├── entry.S
│   ├── fcntl.h (新增, 文件相关的一些抽象)
│   ├── file.c (更加完成的文件操作)
│   ├── file.h (更加完成的文件定义)
│   ├── fs.c (新增, 文件系统实际逻辑)
│   ├── fs.h
│   ├── kalloc.c
│   ├── kalloc.h
│   ├── kernel.ld
│   ├── kernelvec.S
│   ├── link_app.S
│   ├── loader.c
│   ├── loader.h
│   ├── log.h
│   ├── main.c
│   ├── plic.c (新增, 用来处理磁盘中断)
│   ├── plic.h (新增, 用来处理磁盘中断)
│   ├── printf.c
│   ├── printf.h
│   ├── proc.c
│   ├── proc.h
│   ├── riscv.h
│   ├── sbi.c
│   ├── sbi.h
│   ├── string.c
│   ├── string.h
│   ├── switch.S
│   └── syscall.c
```

```
|   ├── syscall.h
|   ├── syscall_ids.h
|   ├── timer.c
|   ├── timer.h
|   ├── trampoline.S
|   ├── trap.c
|   ├── trap.h
|   ├── types.h
|   ├── virtio_disk.c (新增, 用来处理磁盘中断)
|   ├── virtio.h (新增, 用来处理磁盘中断)
|   ├── vm.c
|   └── vm.h
└── README.md
└── scripts
    └── initproc.py (弱化的 pack.py, 仅仅用来插入 INIT_PROC 符号)
└── user
```

本章代码导读

本章涉及的代码量相对较多，且与进程执行相关的管理还有直接的关系。其实我们是参考经典的UNIX基于索引的文件系统，设计了一个简化的有一级目录并支持创建/打开/读写/关闭文件一系列操作的文件系统，也就是说本章。本章采用的文件系统和ext4文件系统比较类似。其中也涉及到了inode这个概念。进入本章之后，我们的测例文件一开始是存放在我们生成的“磁盘”上的，需要我们实现磁盘的读写来进行操作了。我们实现了一个简单的 nfs 文件系统，具体的结构将在下面的章节中说明。大家可以看一看我们本章对 makefile 文件的改动。

```
QEMU = qemu-system-riscv64
QEMUOPTS = \
    -nographic \
    -smp $(CPUS) \
    -machine virt \
    -bios $(BOOTLOADER) \
    -kernel kernel \
+   -drive file=$(U)/fs.img,if=none,format=raw,id=x0 \           # 以 user/fs.img 作为磁盘镜像
+   -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0     # 虚拟 virtio 磁盘设备
```

在操作系统层面，我们的读写文件操作都是在内核态中完成的。由于读写磁盘所需的时间并不确定，所以我们需要一种新的中断机制来通知操作系统“读写已完成”——这种机制就是外部中断。这意味着，如果要在内核态处理中断，就需要短暂允许中断嵌套（即在处理一个中断时，允许另一个中断发生）。

注：中断嵌套指的是在内核执行过程中，如果发生中断，可以暂停当前操作去处理新的中断，处理完后再回来继续执行原来的操作。这在处理异步事件（如磁盘I/O完成）时非常有用。

一旦操作系统成功打开了某个文件，我们就能获得一个对应的文件描述符（fd）。

注：文件描述符（fd）是一个整数，用于在后续操作中唯一标识这个已打开的文件。

有了这个fd，我们就可以通过系统调用 `sys_write` 或 `sys_read` 来读写该文件了。

注：在lab6中，我们其实已经实现了类似的功能，因此这里可以类比理解。

文件系统接口

本节导读

本节中，我们将以Linux系统中的常规文件和目录为例，从应用程序访问文件的角度，介绍文件的一些重要特点及基本使用方法。由于Linux的文件系统模型较为复杂，我们在自己的内核中对其进行了大幅简化。我们也会说明具体做了哪些简化。

最后，我们会讲解：在本系统上开发应用程序时，应如何使用我们简化后的文件系统，以及相关的必要知识。

文件和目录

常规文件

在操作系统用户看来，常规文件是存储在持久性设备（如硬盘）中的一个字节序列。每个常规文件都有一个文件名（Filename），用户通过文件名来区分不同的文件。

为了方便描述，以下内容中的“文件”一词，有时可能指常规文件、目录，也可能指之前提到过的标准输入、标准输出、管道等I/O资源。请读者根据上下文判断具体含义。

在Linux系统上，我们可以使用 `stat` 工具来获取文件的基本信息。下面以项目中的一个源代码文件 `os/main.c` 为例进行演示：

```
$ stat os/main.c

File: os/main.c
Size: 491          Blocks: 8          IO Block: 4096   regular file
Device: 805h/2053d Inode: 4726542      Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1000/deathwish)  Gid: ( 1000/deathwish)
Access: 2021-09-08 17:52:06.915389371 +0800
Modify: 2021-09-08 17:52:06.127425836 +0800
Change: 2021-09-08 17:52:06.127425836 +0800
Birth: -
```

上面的输出展示了 `main.c` 文件的多个信息项，下面逐一解释：

1. File: 表示文件名，这里是 `main.c`。
2. Size: 表示文件大小，单位为字节，这里是 491 字节。
3. Blocks: 表示文件占用的磁盘块数量，这里是 8 个块。

文件系统中，文件数据是以块为单位存储的。从 IO Block 项可以看出，在当前 Ubuntu 系统中，每个块的大小是 4096 字节。

4. regular file: 表示这是一个常规文件。实际上，其他类型的文件（如设备文件、目录等）也可以通过文件名访问。
5. Device: 对于特殊文件（如块设备、字符设备文件），这里会显示其主设备号和次设备号。对于常规文件，我们一般不需要关注此项。
6. Inode: 表示文件的索引节点号。

在文件系统底层，并不是直接用文件名查找文件，而是先将文件名转换为索引节点号，再通过该号找到文件内容。普通用户通常不需要关心这个信息。

7. Links: 表示文件的硬链接数量。

如果在同一个文件系统中，两个文件（目录也是文件）拥有相同的索引节点号，它们就是硬链接关系。Links 的值其实就是该文件拥有的不同文件名的数量。本章练习中，你将有机会在文件系统中实现硬链接功能。

8. Uid 和 Gid: 分别表示文件所属的用户 ID 和用户组 ID。

Access 项中的一串字符（例如 `-rw-r--r--`）也是一种权限表示方式。第一位表示文件类型（`-` 代表常规文件，`d` 代表目录等）。后面 9 位分为三组，分别表示文件所有者、所属组内其他用户、其他所有用户对该文件的读、写、执行权限。

9. Access 和 Modify: 分别表示文件的最近访问时间和最近修改时间。

通常，我们可以通过文件扩展名来推测文件用途，比如 `main.c` 的扩展名 `.c` 表示它是一个 C 语言源代码文件。但对操作系统内核而言，所有文件都被视为一个无差别的字节序列，文件内容的结构和含义由对应的应用程序来解析。

目录

早期的文件系统仅通过文件名区分文件，这在文件归档和管理上存在困难。现在，我们习惯将文件按功能或属性分类，存放在不同层级的目录中，从而更容易逐级查找文件。结合用户和用户组的概念，目录也便于权限管理：通过对目录设置权限，可以间接控制用户或用户组对其内部文件的访问，这增强了操作系统在多用户环境下的安全性。

同样可以用 `stat` 查看目录信息：

```
$ stat os

File: os
Size: 4096      Blocks: 8      IO Block: 4096   directory
Device: 805h/2053d    Inode: 4726538    Links: 9
Access: (0775/drwxrwxr-x)  Uid: ( 1000/deathwish)  Gid: ( 1000/deathwish)
Access: 2021-09-08 17:52:06.915389371 +0800
Modify: 2021-09-08 17:52:06.127425836 +0800
Change: 2021-09-08 17:52:06.127425836 +0800
Birth: -
```

输出中的 `directory` 表示 `os` 是一个目录，从权限字符串首位的 `d` 也能看出。对于目录，权限字符 `rwX` 的含义与文件不同：

1. `r` 表示是否允许查看该目录下的文件和子目录列表。
2. `w` 表示是否允许在该目录下创建或删除文件和子目录。
3. `x` 表示是否允许进入或通过该目录。

Blocks 项显示该目录也占用 8 个块存储。实际上，目录在系统中也被视为一种文件，它有自己的索引节点号，其内容保存着若干目录项。每个目录项可以看作一条映射，将文件名或子目录名对应到它们在文件系统中的索引节点号。与普通文件不同的是，用户不能直接编辑目录内容，只能通过创建、删除其下的文件或子目录来间接修改。

引入目录后，文件和目录可以组织成一种叫做“目录树”的有根树结构（不考虑软链接）。树中每个节点是文件或目录，目录下的文件和子目录都是它的子节点。文件是目录树的叶子节点，树的根节点也是一个目录，称为根目录。目录树中每个节点都可以用绝对路径来定位，该路径是从根目录到目标节点所经过的所有目录名，用路径分隔符连接而成。例如在 Linux 中：

根目录的绝对路径是 `/`，路径分隔符也是 `/`。

`main.c` 的绝对路径可能是 `/home/oslab/workspace/UCORE/uCore-Tutorial-2023S/os/main.c`。

`os` 目录的绝对路径可能是 `/home/oslab/workspace/UCORE/uCore-Tutorial-2023S/os`。

注意：具体绝对路径会因系统环境而不同。

绝对路径通常较长，使用不便。日常操作中，我们一般固定在一个工作目录下工作，因此更常用的是相对路径。每个进程都会记录自己当前的工作目录，如果传递给系统的路径不是以 `/` 开头，系统会将其视为相对于当前工作目录的相对路径，并自动拼接成绝对路径后再进行查找。其中：

`./` 表示当前目录。

`../` 表示当前目录的上层目录。

在终端中，可以使用 `pwd` 命令查看当前工作目录，使用 `cd` 命令切换工作目录。

有了目录以后，我们不再仅通过文件名索引文件，而是通过路径（绝对或相对）来索引。在文件系统底层，路径转换为索引节点号的过程是逐级进行的：对于绝对路径，从根目录开始，根据目录内容找到下一级目录的索引节点号，依次向下，直到目标文件或目录。在此过程中，目录的权限设置会起到访问控制作用，防止无权限用户访问。

注解：目录是否有必要存在？

基于路径的索引方式在并行或分布式环境下较难优化，因为查找下一级目录必须先获得当前级目录的内容，这是一个串行过程。在一些对性能要求极高的场景中，可以考虑弱化目录的权限管理功能，将目录树结构扁平化，使文件系统的磁盘布局更接近键值对存储的形式。

文件系统

计算机中常见的文件和目录，实际上都保存在硬盘这类可以长期保存数据的设备中。这些设备（也叫持久存储设备）有一个特点：它们只能按“扇区”为单位进行读取或写入，而且可以随机访问任意一个扇区。但我们在使用计算机时，习惯通过路径（比如 `/home/user/a.txt`）直接找到文件并进行操作，这看起来简单直观。那么，这两者之间是如何转换的呢？中间负责这项工作的部分，就叫做文件系统（File System）。

简单来说，文件系统的任务，就是把我们在逻辑上看到的那个目录树（包含文件、目录以及它们内部的数据等信息），转换成硬盘上一个一个扇区的具体摆放方式。反过来，它也能根据硬盘上已有的扇区数据，重新构造出我们熟悉的那个目录树结构。

可以这样想象：文件系统就像一个仓库管理员。我们用户只需要说“我要a号商品”，管理员就知道这个商品具体放在仓库的哪个货架、哪个位置（对应硬盘的哪个扇区），并帮我们取出来。

世界上存在多种不同的文件系统，它们对同一个目录树可能会有完全不同的存储安排。常见的例子有，Windows 系统常用的 FAT、NTFS，以及 Linux 系统常用的 ext3、ext4 等。

一台计算机里可以有多块硬盘或者多个存储分区，它们可能使用不同的文件系统格式。为了能方便地统一管理它们，操作系统内核中引入了一个中间层，叫做虚拟文件系统（VFS, Virtual File System）。

VFS 相当于制定了一套通用的“语言”或“接口标准”。它规定了目录树应该长什么样，以及“打开文件”、“读取文件”等操作应该怎么定义。任何具体的文件系统（比如 ext4、NTFS），只要按照这套标准实现好对应的功能，再通过“挂载（Mount）”的方式接入系统，就可以被统一到一棵逻辑目录树下进行管理了。这样，用户和程序就不用关心底层具体是哪种文件系统了。

简易文件与目录抽象

为了清晰地展示文件系统的基本工作原理，同时又不让代码过于复杂，我们在这个教学用的内核实现中，对目录树结构做了很多简化。简化点主要包括：

1. 扁平化结构：系统中只有一个目录，就是根目录 `/`。所有的文件都直接放在根目录下。因此，要找一个文件时，直接使用它的文件名即可，不需要使用像 `/home/a.txt` 这样带路径的名字。

2. 无权限控制：系统不区分用户和用户组，只有一个用户。同时，根目录和所有文件都没有设置读、写、执行等权限控制位，意味着对文件的任何操作都不会被禁止。
3. 无时间戳：不记录文件的创建时间、最后访问时间或最后修改时间。
4. 不支持链接：既不支持软链接（快捷方式），也不支持硬链接。
5. 系统调用简化：除了下面会介绍的几个必要系统调用外，很多其他与文件系统相关的系统调用都没有实现。

打开与读写文件的系统调用

文件打开

在读取或写入一个普通文件之前，应用程序首先要通过内核提供的 `sys_open` 系统调用“打开”这个文件。这个过程就像是告诉操作系统：“我准备要操作这个文件了，请帮我准备好。”

操作系统收到请求后，会在这个进程的“文件描述符表”中为这个文件预留一个位置（称为一个表项），然后返回一个整数给应用程序。这个整数就叫做文件描述符，它其实就是刚才那个表项在表格里的编号（索引值）。后续对这个文件的所有操作，都需要使用这个文件描述符来进行：

```
/// 功能：打开一个常规文件，并返回可以访问它的文件描述符。
/// 参数：path 描述要打开的文件的文件名（简单起见，文件系统不需要支持目录，所有的文件都放在根目录 / 下），
/// flags 描述打开文件的标志，具体含义下面给出。
/// 返回值：如果出现了错误则返回 -1，否则返回打开常规文件的文件描述符。可能的错误原因是：文件不存在。
/// syscall ID: 56
int open(int dirfd, char* path, unsigned int flags, unsigned int mode);
```

目前我们的内核支持以下几种标志（多种不同标志可能共存）：

1. 如果 `flags` 为 0，则表示以只读模式 `RONLY` 打开；
2. 如果 `flags` 的第 0 位被设置（值为 `0x001`），表示以只写模式 `WONLY` 打开；
3. 如果 `flags` 的第 1 位被设置（值为 `0x002`），表示以可读可写模式 `RW` 打开；
4. 如果 `flags` 的第 9 位被设置（值为 `0x200`），表示允许创建文件 `CREATE`。此时如果找不到该文件，就应当创建该文件；如果文件已经存在，则应当把该文件的大小清零；
5. 如果 `flags` 的第 10 位被设置（值为 `0x400`），则在打开文件时需要清空文件内容，并将文件大小归零，这个标志叫做 `TRUNC`。本章我们暂时不会用到这个标志。

注意：`flags` 中的权限设置只能控制本次打开后进程对该文件的访问方式。在实际的文件系统中，打开文件前通常还需要检查文件自身的访问权限，比如一个文件本身不允许写入，那么即使进程尝试用 `WONLY` 或 `RW` 标志去打开也会失败。不过在我们简化的文件系统中，文件没有设置权限控制，因此这一步可以跳过。

文件的顺序读写

打开文件并获得文件描述符 `fd` 之后，我们就可以用之前介绍过的 `sys_read` 和 `sys_write` 两个系统调用来对它进行读写。需要注意的是，常规文件的读写模式与我们之前介绍的几种文件类型有所不同。标准输入输出、匿名管道都属于流式读写，而常规文件则同时支持顺序读写和随机读写。

由于常规文件可以看作一段连续的字节序列，我们理应能够任意读写其中任何一段数据，这就是随机读写。但只靠 `sys_read` 和 `sys_write` 两个系统调用是无法实现这一点的。如果你之前使用过 C 语言，就会知道读写文件时总是有一个偏移量，下一次读写的起始位置就是由上一次读写结束的位置决定的。我们可以通过 `lseek` 函数来改变这个偏移量（本章暂不需要实现）。

补充一点：在文件系统的底层实现中，其实都是基于随机读写的方式进行的。

nfs文件系统

本节导读

本节我们简单介绍一下本章实现的 nfs 文件系统。本章新增的代码量比较大，但是大部分函数通过名称就能理解其功能，理解起来并不困难，不需要逐行深入代码细节。

文件系统布局

在导言中提到，nfs 文件系统的设计与 ext4 文件系统非常相似，下面我们来看一下 nfs 文件系统的布局情况：

```
// 基本信息：块大小 BSIZE = 1024B，总容量 FSSIZE = 1000 个 block = 1000 * 1024 B。  
// Layout: // 0号块留待后续拓展，可以忽略。superblock 固定为 1 号块，size 固定为一个块。  
// 其后是储存 inode 的若干个块，占用块数 = inode 上限 / 每个块上可以容纳的 inode 数量，  
// 其中 inode 上限固定为 200，每个块的容量 = BSIZE / sizeof(struct disk_inode)  
// 再之后是数据块相关内容，包含一个 储存空闲块位置的 bitmap 和 实际的数据块，bitmap 块  
// 数量固定为 NBITMAP = FSSIZE / (BSIZE * 8) + 1 = 1000 / 8 + 1 = 126 块。  
// [ boot block | sb block | inode blocks | free bit map | data blocks ]
```

注意：不推荐同学们修改该布局，除非你完全看懂了 fs 的逻辑，所以最好不要改变 `disk_inode` 这个结构的大小，如果想要增删字段，一定使用 `pad`。这个布局具体定义的位置在 `nfs/fs.c` 之中。

我们定义的 `inode`（索引节点）和 `data blocks`（数据块）与 ext4 文件系统中同名的结构功能基本一致。索引节点是文件系统中的核心数据结构，用于记录文件和目录的完整信息。

在逻辑目录树中，每个文件或目录都对应一个独立的 inode。前面提到的文件/目录底层编号，指的就是 inode 编号。每个 inode 不仅保存着通过 `stat` 命令可以查看的元数据信息，例如文件大小、访问权限、文件类型等，还包含指向实际数据存储位置的信息——这些信息用于定位文件/目录内容存放在磁盘的哪些数据块中。

元数据指的是描述数据的数据，比如文件的创建时间、修改时间、所有者等信息，它们并不直接包含文件的实际内容。

在数据组织方式上，我们的 inode 同时支持直接索引与间接索引机制。

直接索引是指 inode 中直接存放数据块的地址；间接索引则是通过多级指针间接查找数据块，常用于存储较大的文件。

接下来我们来看一下它们在 C 语言中对应的具体结构体定义：

```
// 超级块位置固定，用来指示文件系统的一些元数据，这里最重要的是 inodestart 和 bmapstart  
struct superblock {  
    uint magic;        // Must be FSMAGIC  
    uint size;         // Size of file system image (blocks)  
    uint nblocks;      // Number of data blocks  
    uint ninodes;      // Number of inodes.  
    uint inodestart;   // Block number of first inode block  
    uint bmapstart;    // Block number of first free map block  
};  
  
// On-disk inode structure  
// 储存磁盘 inode 信息，主要是文件类型和数据块的索引，其大小影响磁盘布局，不要乱改，可以用 pad
```



```

struct dinode {
    short type;           // File type
    short pad[3];
    uint size;            // Size of file (bytes)
    uint addrs[NDIRECT + 1]; // Data block addresses
};

// in-memory copy of an inode
// dinode 的内存缓存，为了方便，增加了 dev, inum, ref, valid 四项管理信息，大小无所谓，可以随便改。
struct inode {
    uint dev;             // Device number
    uint inum;            // Inode number
    int ref;              // Reference count
    int valid;            // inode has been read from disk?
    short type;           // copy of disk inode
    uint size;
    uint addrs[NDIRECT+1]; // data block num
};

// 目录对应的数据块的内容本质是 filename 到 file inode_num 的一个 map，这里为了简单，就存为一个
`dirent` 数组，查找的时候遍历对比
struct dirent {
    ushort inum;
    char name[DIRSIZ];
};

// 数据块缓存结构体。
struct buf {
    int valid;    // has data been read from disk?
    int disk;     // does disk "own" buf?
    uint dev;
    uint blockno;
    uint refcnt;
    struct buf *prev; // LRU cache list
    struct buf *next;
    uchar data[BSIZE];
};

```

注意几个量的概念:

1. block num: 表示某一个磁盘块的编号。我们操作数据块会把它读入内存的数据块缓存之中，其结构体见上。
2. inode num: 表示某一个 inode 在所有 inode 项里的编号。注意 inode blocks 其实就是一个 inode 的大数组。

同时，目录本身是一个 filename 到 file对应的inode_num的map，可以完成 filename 到 inode_num 的转化。

OS启动后是没有inode的内存缓存的。下面我们自底向上走一遍OS打开已存在在磁盘上文件的过程，让大家熟悉一下 nfs的具体实现方式。

virtio 磁盘驱动

注意：这部分代码的细节同学们不必完全掌握，但需要了解大致的流程。

在 uCore-Tutorial 中，读写磁盘块是通过中断方式完成的。

我们在 `virtio.h` 和 `virtio-disk.c` 文件中依照 qemu 对 virtio 的定义，实现了 `virtio_disk_init` 和 `virtio_disk_rw` 这两个函数。

其中，`virtio_disk_init` 完成磁盘设备本身的初始化以及相关管理结构的初始化；

`virtio_disk_rw` 则负责实际的磁盘 IO 操作。

具体来说，当设置好读写信息之后，会通过 MMIO 的方式通知磁盘开始工作。

然后操作系统会开启中断，并进入等待状态，直到磁盘完成读写。

磁盘完成 IO 后会触发一个外部中断，中断处理程序会解除等待循环。

需要注意的是，内核只在处理磁盘读写时短暂打开中断，完成后立即关闭。

```
virtio_disk_rw(struct buf *b, int write) {
    /// ... set IO config
    *R(VIRTIO_MMIO_QUEUE_NOTIFY) = 0;           // notify the disk to carry out IO
    struct buf volatile * _b = b;               // Make sure compiler will load 'b' from
memory
    intr_on();
    while(_b->disk == 1); // _b->disk == 0 means that this IO is done
    intr_off();
}

// 开启和关闭中断的函数。
static inline void intr_on() { w_sstatus(r_sstatus() | SSTATUS_SIE); }

// disable device interrupts
static inline void intr_off() { w_sstatus(r_sstatus() & ~SSTATUS_SIE); }
```

对于内核中断处理的修改在 `trap.c` 之中。之前我们的 trap from kernel 会直接 panic，现在我们需要添加对外部中断的处理。kerneltrap 也需要类似 usertrap 的保存上下文以及回到原处的 `kernelvec` 以及 `kernelret` 函数。进入内核之后要单独设置 stvec 指向 kernelvec 处。

```
# kernelvec.S

kernelvec:
    // make room to save registers.
    addi sp, sp, -256
    // save the registers expect x0
    sd ra, 0(sp)
    sd sp, 8(sp)
    sd gp, 16(sp)
    // ...
    sd t4, 224(sp)
    sd t5, 232(sp)
    sd t6, 240(sp)

    call kerneltrap
```

```

kernelret:
    // restore registers.
    // 思考：为什么直接就使用了sp?
    ld ra, 0(sp)
    ld sp, 8(sp)
    ld gp, 16(sp)
    // restore all registers expect x0
    ld t4, 224(sp)
    ld t5, 232(sp)
    ld t6, 240(sp)
    addi sp, sp, 256
    sret

```

kerneltrap 具体的修改如下:

```

void kerneltrap() {
    // 老三样，不过在这里把处理放到了 c 代码中
    uint64 sepc = r_sepc();
    uint64 sstatus = r_sstatus();
    uint64 scause = r_scause();

    if ((sstatus & SSTATUS_SPP) == 0)
        panic("kerneltrap: not from supervisor mode");

    if (scause & (1ULL << 63)) {
        // 可能发生时钟中断和外部中断，我们的主要目标是处理外部中断
        devintr(scause & 0xff);
    } else {
        // kernel 发生异常就挣扎了，肯定出问题了，杀掉用户线程跑路
        error("invalid trap from kernel: %p, stval = %p sepc = %p\n", scause, r_stval(),
sepc);
        exit(-1);
    }
}

// 外部中断处理函数
void devintr(uint64 cause) {
    int irq;
    switch (cause) {
        case SupervisorTimer:
            set_next_timer();
            // 时钟中断如果发生在内核态，不切换进程，原因分析在下面
            // 如果发生在用户态，照常处理
            if ((r_sstatus() & SSTATUS_SPP) == 0) {
                yield();
            }
            break;
        case SupervisorExternal:
            irq = plic_claim();
            if (irq == UART0_IRQ) { // UART 串口的终端不需要处理，这个 rustsbi 替我们处理
好了
                // do nothing
            } else if (irq == VIRTIO0_IRQ) { // 我们等的就是这个中断

```

```

        virtio_disk_intr();
    }
    if (irq)
        plic_complete(irq);          // 表明中断已经处理完毕
    break;
}
}

```

`virtio_disk_intr()` 会把 `buf->disk` 置零，这样中断返回后死循环条件解除，程序可以继续运行。具体代码在 `virtio-disk.c` 中。

这里还有一个关键点需要注意：为什么在内核中始终不允许进行进程切换？这主要是因为我们的内核尚未支持并发，相关数据结构没有使用锁或其他同步机制进行保护。假设一种情况：一个进程正在读写某个文件，而内核在处理磁盘响应时进入等待状态；此时如果发生时钟中断并切换到另一个进程，而该进程也试图读写同一个文件，就可能引发数据访问冲突，甚至导致磁盘出现错误操作。因此，这也是为什么在内核态下始终不处理时钟中断的原因——我们必须确保每一次内核操作都是原子性的，不可被打断。

大家可以思考一下：如果内核可以随时切换，哪些数据结构可能会被破坏？例如：`kalloc` 进行到一半时被中断，或者进程 `switch` 切换过程中被强行打断等情况。

磁盘块缓存

为了加快磁盘访问的速度，在内核中设置了磁盘缓存 `struct buf`，一个 `buf` 对应一个磁盘 `block`，这一部分代码也不要求同学们深入掌握。大致的作用机制是，对磁盘的读写都会被转化为对 `buf` 的读写，当 `buf` 有效时，读写 `buf`，`buf` 无效时（类似页表缺页和 TLB 缺失），就实际读写磁盘，将 `buf` 变得有效，然后继续读写 `buf`。详细的内容在 `buf.h` 和 `bio.c` 中。`buf` 写回的时机是 `buf` 池满需要替换的时候(类似内存的 `swap` 策略) 手动写回。如果 `buf` 没有写回，一旦掉电就 GG 了，所以手动写回还是挺重要的。

```

// os/bio.c
struct buf *
bread(uint dev, uint blockno) {
    struct buf *b;
    b = bget(dev, blockno);
    if (!b->valid) {
        virtio_disk_rw(b, R);
        b->valid = 1;
    }
    return b;
}

// write b's contents to disk.
void bwrite(struct buf *b) {
    virtio_disk_rw(b, W);
}

```

读取文件数据实际就是读取文件 `inode` 指向数据块的数据。读数据块到缓存的数据需要使用 `bread`，而写回缓存需要用到 `bwrite` 函数。文件系统首先使用 `bget` 去查缓存中是否已有对应的 `block`，如果没有会分配内存来缓存对应的块。之后会调用 `bread / bwrite` 进行从磁盘读数据块、写回数据块。要注意释放块缓存的 `brelse` 函数。

```
// os/bio.c
void brelse(struct buf *b) {
    b->refcnt--;
    if (b->refcnt == 0) {
        b->next->prev = b->prev;
        b->prev->next = b->next;
        b->next = bcache.head.next;
        b->prev = &bcache.head;
        bcache.head.next->prev = b;
        bcache.head.next = b;
    }
}
```

需要特别注意的是 `brelse` 不会真的如字面意思释放一个 `buf`。它的准确含义是暂时不操作该 `buf` 了并把它放置在 `bcache` 链表的首部，`buf` 的真正释放会被推迟到 `buf` 池满，无法分配的时候，就会把最近最久未使用的 `buf` 释放掉（释放 = 写回 + 清空）。这是为了尽可能保留内存缓存，因为读写磁盘真的太太太慢了。

此外，`brelse` 的数量必须和 `bget` 相同，因为 `bget` 会使得引用计数加一。如果没有相匹配的 `brelse`，就好比 `new` 了之后没有 `delete`。千万注意。

inode的操作

现在来看看 `nfs` 如何读取磁盘上的 `dinode` 到内存之中。我们通过 `file name` 对应的 `inode num` 去从磁盘读取对应的 `inode`。为了解决共享问题（不同进程可以打开同一个磁盘文件），也有一个全局的 `inode table`，每当新打开一个文件的时候，会把一个空闲的 `inode` 绑定为对应 `dinode` 的缓存，这一步通过 `iget` 完成。

```
// 找到 inum 号 dinode 绑定的 inode, 如果不存在新绑定一个
static struct inode *iget(uint dev, uint inum) {
    struct inode *ip, *empty;
    // 遍历查找 inode table
    for (ip = &itable.inode[0]; ip < &itable.inode[NINODE]; ip++) {
        // 如果有对应的，引用计数 +1并返回
        if (ip->ref > 0 && ip->dev == dev && ip->inum == inum) {
            ip->ref++;
            return ip;
        }
    }
    // 如果没有对应的，找一个空闲 inode 完成绑定
    empty = find_empty();
    // GG, inode 表满了，果断自杀.lab7正常不会出现这个情况。
    if (empty == 0)
        panic("iget: no inodes");
    // 注意这里仅仅是写了元数据，没有实际读取，实际读取推迟到后面
    ip = empty;
    ip->dev = dev;
    ip->inum = inum;
    ip->ref = 1;
    ip->valid = 0; // 没有实际读取, valid = 0
    return ip;
}
```

当已经得到一个文件对应的 `inode` 后, 可以通过 `ivalid` 函数确保其是有效的。

```
// Reads the inode from disk if necessary.
void ivalid(struct inode *ip) {
    struct buf *bp;
    struct dinode *dip;
    if (ip->valid == 0) {
        // bread 可以完成一个块的读取, 这个在将 buf 的时候说过了
        // IBLOCK 可以计算 inum 在几个 block
        bp = bread(ip->dev, IBLOCK(ip->inum, sb));
        // 得到 dinode 内容
        dip = (struct dinode *) bp->data + ip->inum % IPB;
        // 完成实际读取
        ip->type = dip->type;
        ip->size = dip->size;
        memmove(ip->addrs, dip->addrs, sizeof(ip->addrs));
        // buf 暂时没用了
        brelse(bp);
        // 现在有效了
        ip->valid = 1;
    }
}
```

在 `inode` 有效之后, 可以通过 `writei`, `readi` 完成读写。这又是 `bwrite` 和 `bread` 的上级接口了。和其他OS支持的文件系统一样, 我们首先计算出文件的偏移量, 并通过 `bmap` 得到对应的 `block num`。之后调用 `bwrite`/`bread` 来进行文件的读写操作。

```
// 从 ip 对应文件读取 [off, off+n) 这一段数据到 dst
int readi(struct inode *ip, char* dst, uint off, uint n) {
    uint tot, m;
    // 还记得 buf 吗?
    struct buf *bp;
    for (tot = 0; tot < n; tot += m, off += m, dst += m) {
        // bmap 完成 off 到 block num 的对应, 见下
        bp = bread(ip->dev, bmap(ip, off / BSIZE));
        // 一次最多读一个块, 实际读取长度为 m
        m = MIN(n - tot, BSIZE - off % BSIZE);
        memmove(dst, (char*)bp->data + (off % BSIZE), m);
        brelse(bp);
    }
    return tot;
}

// 同 readi
int writei(struct inode *ip, char* src, uint off, uint n) {
    uint tot, m;
    struct buf *bp;

    for (tot = 0; tot < n; tot += m, off += m, src += m) {
        bp = bread(ip->dev, bmap(ip, off / BSIZE));
        m = MIN(n - tot, BSIZE - off % BSIZE);
        memmove(src, (char*)bp->data + (off % BSIZE), m);
    }
}
```



```

        bwrite(bp);
        brelse(bp);
    }

    // 文件长度变长, 需要更新 inode 里的 size 字段
    if (off > ip->size)
        ip->size = off;

    // 有可能 inode 信息被更新了, 写回
    iupdate(ip);

    return tot;
}

```

其中 bmap 函数是连接 inode 和 block 的重要函数。但由于我们支持了间接索引，同时还涉及到文件大小的改变，所以也拉出来看看：

```

// bn = off / BSIZE
uint bmap(struct inode *ip, uint bn) {
    uint addr, *a;
    struct buf *bp;
    // 如果 bn < 12, 属于直接索引, block num = ip->addr[bn]
    if (bn < NDIRECT) {
        // 如果对应的 addr, 也就是 block num = 0, 表明文件大小增加, 需要给文件分配新的 data block
        // 这是通过 balloc 实现的, 具体做法是在 bitmap 中找一个空闲 block, 置位后返回其编号
        if ((addr = ip->addrs[bn]) == 0)
            ip->addrs[bn] = addr = balloc(ip->dev);
        return addr;
    }
    bn -= NDIRECT;
    // 间接索引块, 那么对应的数据块就是一个大 addr 数组。
    if (bn < NINDIRECT) {
        // Load indirect block, allocating if necessary.
        if ((addr = ip->addrs[NDIRECT]) == 0)
            ip->addrs[NDIRECT] = addr = balloc(ip->dev);
        bp = bread(ip->dev, addr);
        a = (uint *) bp->data;
        if ((addr = a[bn]) == 0) {
            a[bn] = addr = balloc(ip->dev);
            bwrite(bp);
        }
        brelse(bp);
        return addr;
    }

    panic("bmap: out of range");
    return 0;
}

```

balloc(位于 nfs/fs.c)会分配一个新的 buf 缓存。而 iupdate 函数则是把修改之后的 inode 重新写回到磁盘上。不然掉电了就凉了。

```

// Copy a modified in-memory inode to disk.
// Must be called after every change to an ip->xxx field
// that lives on disk.
void iupdate(struct inode *ip) {
    struct buf *bp;
    struct dinode *dip;

    bp = bread(ip->dev, IBLOCK(ip->inum, sb));
    dip = (struct dinode *) bp->data + ip->inum % IPB;
    dip->type = ip->type;
    dip->size = ip->size;
    memmove(dip->addrs, ip->addrs, sizeof(ip->addrs));
    bwrite(bp);
    brelse(bp);
}

```

文件在进程中的结构

在操作系统中，`inode` 是磁盘上 `dinode` 结构在内存中的映射，它由操作系统统一管理。但是，当一个进程使用某个文件时，除了要知道该文件对应哪个 `inode` 之外，还需要记录这个文件在当前进程中被如何使用的一些具体信息。因此，在进程内部，我们使用一个叫做 `file` 的结构体来表示一个正在被该进程使用的文件。

注释：你可以把 `inode` 理解为文件的“身份证”，它记录文件的元信息（如大小、权限、位置等），而 `file` 结构体就像是进程手里拿着的“使用说明书”，记载进程如何操作这个文件（例如读写位置、打开模式等）。

```

// Defines a file in memory that provides information about the current use of the file and
// the corresponding inode location
struct file {
    enum { FD_NONE = 0, FD_INODE, FD_STDIO } type;
    int ref; // reference count
    char readable;
    char writable;
    struct inode *ip; // FD_INODE
    uint off;
};

struct file filepool[FILEPOOLSIZ];

```

我们采用预分配的方式来对 `file` 进行分配，每一个需要使用的 `file` 都要与 `filepool` 中的某一个 `file` 完成绑定。`file` 结构中，`ref` 记录了其引用次数，`type` 表示了文件的类型，在本章中我们主要使用 `FD_NONE` 和 `FD_INODE` 属性，其中 `FD_INODE` 表示 `file` 已经绑定了一个文件（可能是目录或普通文件），`FD_NONE` 表示该 `file` 还没完成绑定，`FD_STDIO` 用来做标准输入输出，这里不做讨论；`readable` 和 `writable` 规定了进程对文件的读写权限；`ip` 标识了 `file` 所对应的磁盘中的 `inode` 编号，`off` 即文件指针，用作记录文件读写时的偏移量。

分配文件时，我们从 `filepool` 中寻找还没有被分配的 `file` 进行分配：

```
// os/file.c
struct file* filealloc() {
    for(int i = 0; i < FILE_MAX; ++i) {
        if(filepool[i].ref == 0) {
            filepool[i].ref = 1;
            return &filepool[i];
        }
    }
    return 0;
}
```

进程关闭文件时，也要去 filepool 中放回：（注意需要根据 ref 来判断是否需要回收该 file）

```
void fileclose(struct file *f)
{
    if (f->ref < 1)
        panic("fileclose");
    if (--f->ref > 0) {
        return;
    }
    switch (f->type) {
        case FD_STDIO:
            // Do nothing
            break;
        case FD_INODE:
            iput(f->ip);
            break;
        default:
            panic("unknown file type %d\n", f->type);
    }

    f->off = 0;
    f->readable = 0;
    f->writable = 0;
    f->ref = 0;
    f->type = FD_NONE;
}
```

注意文件对于进程而言也是其需要记录的一种资源，因此我们在进程对应的PCB结构体之中也需要记录进程打开的文件信息。我们给PCB增加文件指针数组。

```
// proc.h
// Per-process state
struct proc {
    // ...

+   struct file* files[16];
};

// os/proc.c
int fdalloc(struct file* f) {
```

```

struct proc* p = curr_proc();
// fd = 0,1,2 is reserved for stdio/stdout/stderr
for(int i = 3; i < FD_MAX; ++i) {
    if(p->files[i] == 0) {
        p->files[i] = f;
        return i;
    }
}
return -1;
}

```

一个进程能打开的文件是有限的（我们设置为16）。一个进程如果要打开某一个文件，其文件指针数组必须有空位。如果有，就把下标做为文件的 `fd`，并把指定文件指针存入数组之中。

获取文件对应的inode

我们已经知道，文件与inode之间存在对应关系。那么，在实际操作中，如何获取一个文件对应的inode呢？

之前提到过，这个过程是通过查询文件名（file name）与inode的对应关系来实现的。在系统中，这个功能是由目录（directory）来提供的。下面，我们来看一下代码具体是如何实现这个过程的。

首先用户程序要打开指定文件名文件，发起系统调用 `sys_openat`：

打开文件的方式根据 `flags` 有很多种。我们先来看最简单的，就是打开已经存在的文件的方法。`fileopen` 在处理这类打开时调用了 `namei` 这个函数。

```

// namei = 获得根目录，然后在其中遍历查找 path
struct inode *namei(char *path) {
    struct inode *dp = root_dir();
    return dirlookup(dp, path, 0);
}

// root_dir 位置固定
struct inode *root_dir() {
    struct inode* r = iget(ROOTDEV, ROOTINO);
    ivalid(r);
    return r;
}

// 遍历根目录所有的 dirent，找到 name 一样的 inode。
struct inode *dirlookup(struct inode *dp, char *name, uint *poff) {
    uint off, inum;
    struct dirent de;
    // 每次迭代处理一个 block，注意根目录可能有多个 data block
    for (off = 0; off < dp->size; off += sizeof(de)) {
        readi(dp, 0, (uint64) &de, off, sizeof(de));
        if (strncmp(name, de.name, DIRSIZ) == 0) {
            if (poff)
                *poff = off;
            inum = de.inum;
            // 找到之后，绑定一个内存 inode 然后返回
            return iget(dp->dev, inum);
        }
    }
}

```

```

    return 0;
}

```

由于我们是单目录结构。因此首先我们调用 `root_dir` 获取根目录对应的 `inode`。之后就遍历这个 `inode` 索引的数据块中存储的文件信息到 `dirent` 结构体之中，比较名称和给定的文件名是否一致。`dirlookup` 的逻辑对于我们本章的练习十分重要。

`fileopen` 还可能会导致文件 `truncate`，也就是截断，具体做法是舍弃全部现有内容，释放 `inode` 所有 `data block` 并添加到 `free bitmap` 里。这也是目前 `nfs` 中唯一的文件变短方式。

比较复杂的就是使用 `fileopen` 以创建的方式打开一个文件。`fileopen` 函数调用了 `create` 这个函数。

```

static struct inode *create(char *path, short type) {
    struct inode *ip, *dp;
    if(ip = namei(path) != 0) {
        // 已经存在，直接返回
        return ip;
    }
    // 创建一个文件,首先分配一个空闲的 disk inode, 绑定内存 inode 之后返回
    ip = ialloc(dp->dev, type);
    // 注意 ialloc 不会执行实际读取, 必须有 ivalid
    ivalid(ip);
    // 在根目录创建一个 dirent 指向刚才创建的 inode
    dirlink(dp, path, ip->inum);
    // dp 不用了, iput 就是释放内存 inode, 和 iget 正好相反。
    iput(dp);
    return ip;
}

// nfs/fs.c
uint ialloc(ushort type) {
    uint inum = freeinode++;
    struct dinode din;

    bzero(&din, sizeof(din));
    din.type = xshort(type);
    din.size = xint(0);
    winode(inum, &din);
    return inum;
}

// os/fs.c
// write a new directory entry (name, inum) into the directory dp.
int dirlink(struct inode *dp, char *name, uint inum)
{
    int off;
    struct dirent de;
    struct inode *ip;
    // Check that name is not present.
    if((ip = dirlookup(dp, name, 0)) != 0){
        iput(ip);
        return -1;
    }
}

```

```

}

// Look for an empty dirent.
for(off = 0; off < dp->size; off += sizeof(de)){
    if(readi(dp, 0, (uint64)&de, off, sizeof(de)) != sizeof(de))
        panic("dirlink read");
    if(de.inum == 0)
        break;
}
strncpy(de.name, name, DIRSIZ);
de.inum = inum;
if(wrotei(dp, 0, (uint64)&de, off, sizeof(de)) != sizeof(de))
    panic("dirlink");
return 0;
}

```

`ialloc` 干的事情：遍历 inode blocks 找到一个空闲的 inode，初始化并返回。`dirlink` 对于本章的练习也十分重要。和 `dirlookup` 不同，我们没有现成的 `dirent` 存储在磁盘上，而是要在磁盘上创建一个新的 `dirent`。他遍历根目录数据块，找到一个空的 `dirent`，设置 `dirent = {inum, filename}` 然后返回，注意这一步可能找不到空位，这时需要找一个新的数据块，并扩大 `root_dir` size，这是由 `bmap` 自动完成的。需要注意本章创建硬链接时对应 `inode num` 的处理。

文件关闭

文件读写结束后，需要使用 `fclose` 来释放对应的 `inode`，同时也会释放操作系统中的 `file` 结构体与文件描述符 `fd`。实际上，关闭 `inode` 文件只需要调用 `iput` 即可。

注：`iput` 的实现非常简单，本质上只是将 `inode` 的引用计数减一。

你可能会感到疑惑：为什么计数减到 0 时，不立即写回磁盘并释放 `inode`？这与缓冲区 `buf` 的释放机制类似，操作系统会等到 `inode` 池已满时，再自动将其替换出去。这是因为频繁写回磁盘并重新读取的操作速度过慢，会影响整体性能。

务必确保 `iput` 与 `iget` 的调用次数相同，两者必须一一对应。否则可能会导致资源未释放或引用错误等问题。

```

void fileclose(struct file *f)
{
    if(--f->ref > 0) {
        return;
    }
    // 暂时不支持标准输入输出文件的关闭

    if(f->type == FD_INODE) {
        iput(f->ip);
    }

    f->off = 0;
    f->readable = 0;
    f->writable = 0;
    f->ref = 0;
    f->type = FD_NONE;
}

```

```
void iput(struct inode *ip) {
    ip->ref--;
}
```

编程作业：硬链接 (Hard Links)

在理解了 `inode` 与 `dirent` 的分离设计之后，我们自然会想到一个问题：既然一个目录项只是记录了 {文件名, `inode`号} 的映射关系，那么能不能让多个文件名指向同一个 `inode` 呢？

答案是肯定的——这就是**硬链接**的本质。

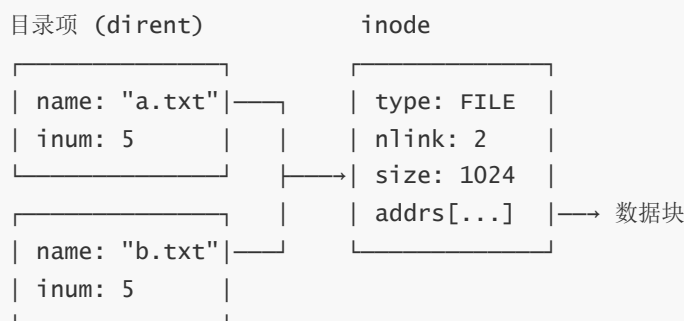
任务目标

本次编程作业需要实现三个系统调用：

1. `sys_linkat`：创建硬链接（为现有文件创建新名称）
2. `sys_unlinkat`：删除硬链接（删除文件的一个名称）
3. `sys_fstat`：获取文件状态信息（包括链接计数）

理解硬链接

回顾一下我们前面学过的文件系统结构：



这张图揭示了硬链接的本质：

- `a.txt` 和 `b.txt` 是两个不同的目录项（不同的名字）
- 但它们的 `inum` 都指向同一个 `inode`（相同的数据）
- `inode` 中的 `nlink` 字段记录了有多少个名称指向它

第一步：发现缺失的 `nlink` 字段

在动手之前，我们需要先检查现有的 `dinode` 和 `inode` 结构：

```
// os/fs.h - 磁盘上的 inode
struct dinode {
    short type;
    short pad[3];
    uint size;
    uint addr[NDIRECT + 1];
}
```



```
};

// os/file.h - 内存中的 inode
struct inode {
    uint dev;
    uint inum;
    int ref;
    int valid;
    short type;
    uint size;
    uint addrs[NDIRECT + 1];
};
```

问题来了：两个结构都没有 `nlink` 字段！要实现硬链接，首先需要添加这个字段。

磁盘 inode (持久化存储)：

```
// os/fs.h
struct dinode {
    short type;
    short nlink;    /* ch6: 硬链接数量 */
    short pad[2];   // 减少一个 pad
    uint size;
    uint addrs[NDIRECT + 1];
};
```

内存 inode (运行时缓存)：

```
// os/file.h
struct inode {
    uint dev;
    uint inum;
    int ref;
    int valid;
    short type;
    short nlink;    /* ch6: 硬链接数量 */
    uint size;
    uint addrs[NDIRECT + 1];
};
```

重要提醒：修改 `dinode` 结构会改变磁盘布局！需要同步修改 `nfs/fs.h` 中的结构定义，并重新生成文件系统镜像。

第二步：同步修改 mkfs 工具

`nfs/fs.c` 负责生成文件系统镜像，其中的 `ialloc()` 函数需要初始化 `nlink`：

```
// nfs/fs.c - ialloc()
uint ialloc(ushort type)
{
    uint inum = freeinode++;
    struct dinode din;

    bzero(&din, sizeof(din));
    din.type = xshort(type);
    din.nlink = xshort(1); /* ch6: 初始化链接计数为1 */
    din.size = xint(0);
    winode(inum, &din);
    return inum;
}
```

第三步：修改 inode 操作函数

在 `os/fs.c` 中，需要在以下位置处理 `nlink`：

分配 inode 时 (`ialloc`)：

```
dip->nlink = 1; /* ch6: 初始化链接数为1 */
```

从磁盘读取时 (`ivalid`)：

```
ip->nlink = dip->nlink; /* ch6: 读取硬链接数 */
```

写回磁盘时 (`iupdate`)：

```
dip->nlink = ip->nlink; /* ch6: 更新硬链接数 */
```

第四步：修改 iput 函数

这是硬链接实现中最关键的修改。原来的 `iput` 只是简单地减少引用计数：

```
void iput(struct inode *ip) {
    ip->ref--;
}
```

但现在我们需要考虑：当最后一个硬链接被删除 (`nlink=0`) 且没有进程打开该文件 (`ref=1`) 时，应该释放 `inode` 及其数据块。

```
// os/fs.c - iput()
void iput(struct inode *ip)
{
    /* ch6: 当引用计数为1且nlink为0时, 释放inode */
    if (ip->ref == 1 && ip->valid && ip->nlink == 0) {
        // 没有目录项指向这个 inode, 且这是最后一个引用
        itrunc(ip);    // 释放数据块
        ip->type = 0;   // 标记为未使用
        iupdate(ip);   // 写回磁盘
        ip->valid = 0;
    }
    ip->ref--;
}
```

第五步：添加 dirunlink 函数

原来的文件系统只有 `dirlink`（添加目录项），但删除硬链接时还需要从目录中移除目录项。参考 `dirlink` 的实现思路：

```
// os/fs.c

/* ch6: 从目录中删除指定名称的目录项 */
int dirunlink(struct inode *dp, char *name)
{
    uint off;
    struct dirent de;
    struct inode *ip;

    /* ch6: 查找目录项 */
    if ((ip = dirlookup(dp, name, &off)) == 0)
        return -1; /* 文件不存在 */

    iput(ip);

    /* ch6: 清空目录项 */
    memset(&de, 0, sizeof(de));
    if (writei(dp, 0, (uint64)&de, off, sizeof(de)) != sizeof(de))
        panic("dirunlink");

    return 0;
}
```

第六步：实现系统调用

`sys_linkat` - 创建硬链接：

```
/* ch6: sys_linkat - 创建硬链接 */
int sys_linkat(int olddirfd, uint64 oldpath_va, int newdirfd,
               uint64 newpath_va, uint flags)
{
    struct proc *p = curr_proc();
```

```

char oldpath[MAX_STR_LEN], newpath[MAX_STR_LEN];
struct inode *ip, *dp;

/* ch6: 从用户空间拷贝路径 */
if (copyinstr(p->pagetable, oldpath, oldpath_va, MAX_STR_LEN) < 0)
    return -1;
if (copyinstr(p->pagetable, newpath, newpath_va, MAX_STR_LEN) < 0)
    return -1;

/* ch6: 新旧路径相同则返回错误 */
if (strcmp(oldpath, newpath, MAX_STR_LEN) == 0)
    return -1;

/* ch6: 查找源文件 */
if ((ip = namei(oldpath)) == NULL)
    return -1;
ivalid(ip);

/* ch6: 不能对目录创建硬链接 */
if (ip->type == T_DIR) {
    iput(ip);
    return -1;
}

/* ch6: 增加链接计数 */
ip->nlink++;
iupdate(ip);

/* ch6: 在根目录中添加新的目录项 */
dp = root_dir();
if (dirlink(dp, newpath, ip->inum) < 0) {
    /* ch6: 添加失败, 恢复链接计数 */
    ip->nlink--;
    iupdate(ip);
    iput(ip);
    iput(dp);
    return -1;
}

iput(dp);
iput(ip);
return 0;
}

```

sys_unlinkat - 删除硬链接:

```

/* ch6: sys_unlinkat - 删除硬链接 */
int sys_unlinkat(int dirfd, uint64 path_va, uint flags)
{
    struct proc *p = curr_proc();
    char path[MAX_STR_LEN];
    struct inode *ip, *dp;

```

```

/* ch6: 从用户空间拷贝路径 */
if (copyinstr(p->pagetable, path, path_va, MAX_STR_LEN) < 0)
    return -1;

/* ch6: 查找文件 */
if ((ip = namei(path)) == NULL)
    return -1;
ivalid(ip);

/* ch6: 不能删除目录（简化实现） */
if (ip->type == T_DIR) {
    iput(ip);
    return -1;
}

/* ch6: 从根目录中删除目录项 */
dp = root_dir();
if (dirunlink(dp, path) < 0) {
    iput(ip);
    iput(dp);
    return -1;
}
iput(dp);

/* ch6: 减少链接计数 */
ip->nlink--;
iupdate(ip);
iput(ip); /* ch6: 如果nlink为0, iput会释放inode */

return 0;
}

```

sys_fstat - 获取文件状态:

```

#define S_IFDIR 0x040000 /* directory */
#define S_IFREG 0x100000 /* regular file */

struct Stat {
    uint64 dev; /* 设备号 */
    uint64 ino; /* inode编号 */
    uint32 mode; /* 文件类型 */
    uint32 nlink; /* 硬链接数量 */
    uint64 pad[7]; /* 填充 */
};

/* ch6: sys_fstat - 获取文件状态 */
int sys_fstat(int fd, uint64 st_va)
{
    struct proc *p = curr_proc();
    struct Stat st;

    /* ch6: 检查fd有效性 */
    if (fd < 0 || fd >= FD_BUFFER_SIZE)

```

```

        return -1;

    struct file *f = p->files[fd];
    if (f == NULL)
        return -1;

    /* ch6: 只支持inode类型的文件 */
    if (f->type != FD_INODE)
        return -1;

    struct inode *ip = f->ip;
    ivalid(ip);

    /* ch6: 填充Stat结构体 */
    memset(&st, 0, sizeof(st));
    st.dev = 0;
    st.ino = ip->inum;
    st.mode = (ip->type == T_DIR) ? S_IFDIR : S_IFREG;
    st.nlink = ip->nlink;

    /* ch6: 拷贝到用户空间 */
    if (copyout(p->pagetable, st_va, (char *)&st, sizeof(st)) < 0)
        return -1;

    return 0;
}

```

常见问题与调试

问题1: ivalid: no type

运行测试时如果出现 `ivalid: no type panic`, 很可能是因为:

- 修改了 `dinode` 结构但没有同步修改 `nfs/fs.h`
- 没有重新生成文件系统镜像

解决方法: 确保三个地方的 `dinode` 定义一致, 然后重新 `make` 生成镜像。

问题2: iget 和 iput 不配对

和前面讲的 `bget/brelse` 一样, `iget` 和 `iput` 必须成对出现。在实现 `sys_linkat` 和 `sys_unlinkat` 时, 需要特别注意错误处理路径上的 `iput` 调用。

nlink 语义总结

操作	nlink 变化
创建文件 (<code>ialloc</code>)	<code>nlink = 1</code>
创建硬链接 (<code>linkat</code>)	<code>nlink++</code>
删除链接 (<code>unlinkat</code>)	<code>nlink--</code>

操作	nlink 变化
nlink = 0 且 ref = 0	释放数据块

文件修改清单

文件	修改内容
os/fs.h	添加 nlink 到 dinode，声明 dirunlink()
os/file.h	添加 nlink 到 inode
os/fs.c	修改 ialloc/ivalid/iupdate/ipt，添加 dirunlink()
os/syscall.c	实现 sys_linkat/sys_unlinkat/sys_fstat
nfs/fs.h	同步修改 dinode 结构
nfs/fs.c	修改 ialloc() 初始化 nlink